

PySpark on Databricks ハンズオン

初級から中級まで:
概念・実装・設計・Delta Lake・性能チューニング

Sparkの基礎知識

Apache Spark とは？

大規模データの分散処理に最適化されたフレームワーク

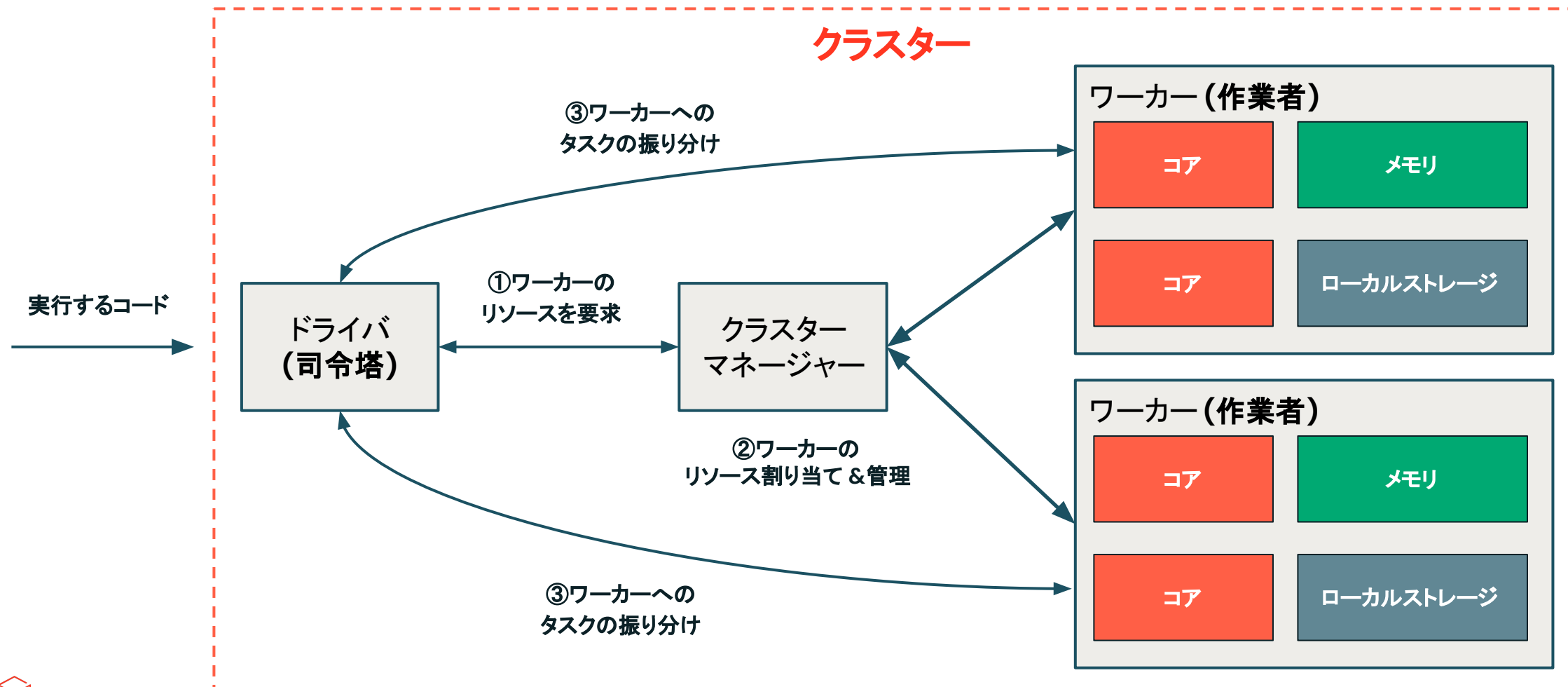


- 統合計算エンジン
- 複数のノードから構成される「クラスター」で並列データ処理
- Sparkはデータの分散処理において最もアクティブに開発されているオープンソースエンジン
- 広く使用されている複数のプログラミング言語をサポート(Python、Java、Scala、R)
- SQLからストリーミング、機械学習に渡る様々なタスクのためのライブラリも提供



Spark の分散処理機構

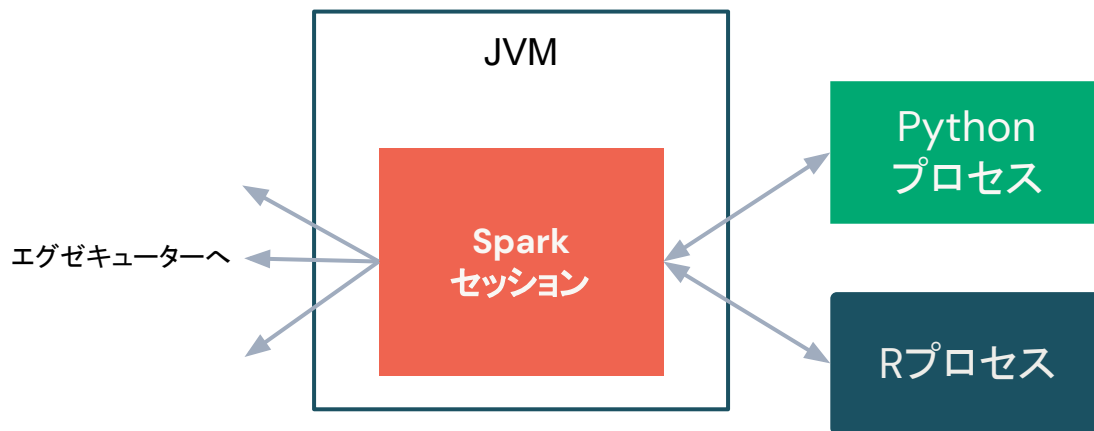
ドライバが分散したタスクをエグゼキュータへ振り分け



Apache Spark とは？

大規模データの分散処理に最適化されたフレームワーク

- SparkSessionはすべてのデータフレームAPIの機能に対する単一のエントリーポイント
- Databricksでは、“spark”という変数名でSparkセッションが自動的に作成されます



PythonでSparkを使用する際には、例えば

```
spark.read.format("データソースのファイル形式")
```

のようにしてデータを読み込む等、様々な操作を行うために使用します。



Spark DataFrame

Sparkで表形式のデータを格納するためのオブジェクト

テーブルと同じように列(カラム)と行(レコード) の概念、カラム名、カラム毎のデータ型、といった情報が保持されている。

item_id	name	price	qty
M_PREM_Q	Premium Queen Mattress	1795	35
M_STAN_F	Standard Full Mattress	945	24
M_PREM_F	Premium Full Mattress	1695	45
M_PREM_T	Premium Twin Mattress	1095	18

以下の例では、データファイルを読み込んで出来たDataFrameが変数"df"に格納される

```
df = spark.read.format("データソースのファイル形式").load("データファイルのパス")
```



DataFrame を起点に様々なデータ変換を行う

列の選択、行のフィルタ、並べ替え、グループ化、集計、列追、etc.

データ変換例 その1

```
df.select("item_id", "price")  
  .where("price > 70")  
  .orderBy("price")
```

1. dfに格納されているDataFrameから"item_id"列と"price"列のデータのみを取り出す (select)
2. "price"列の値が70より大きいレコードのみを抽出 (where)
3. "price"の値が小さい順に並べる (orderBy)

データ変換例 その2

```
df.withColumn("revenue", expr("price * qty"))  
  .groupBy("item_id")  
  .agg(sum("revenue").alias("total_revenue"))
```

1. dfに新たな列"revenue"を追加し、price とqtyを掛けた値を格納する (withColumn)
2. "item_id"でレコードをグループ化 (groupBy)
3. "item_id"毎にrevenueの合計を計算 (agg と sum)
4. 計算した合計を入れる列名を "total_revenue"にする (alias)



Spark は様々なデータソースに対応

データソースからの読み込み & DataFrame の作成

Parquetファイルから作成

```
1  # /path_of_files にある Parquet ファイルから DataFrameを作成
2  df = spark.read.format("parquet").load("/path_of_files")
```

CSVファイルから作成

```
1  # /path_of_files にある csv ファイルから DataFrameを作成
2  df = spark.read.format("csv").load("/path_of_files")
```

同様の書き方で、JSON, Avro, ORC 等からも DataFrameを作成可能



直感的なコーディングでデータ加工

DataFrame を使ったデータ加工

SQLやPandas DataFrameのような感覚で処理を記述可能

Spark内部で最適な実行プランの作成や並列処理化等を行ってくれる

```
1  # user_id毎の販売金額合計を算出
2  new_df = (df
3           .select("user_id", "revenue")
4           .groupBy("user_id")
5           .sum("revenue"))
```

ちなみに・・・Spark は「**遅延評価**」の仕組みになっているため、

上記のコードを実行しただけでは実際のデータ処理は(まだ)行われない



加工したデータを必要な形式で保存

DataFrame からの書き出し

Writeメソッドにフォーマットを指定し、必要な形式で加工したデータを保存

```
1 # new_df を Parquetファイルとして /path_to_save に書き出す
2 (new_df
3  .write
4  .format("parquet")
5  .save("/path_to_save"))
```

「**遅延評価**」の仕組みにより、1つ前のスライドで記述した集計処理は
上記のコードを実行した際に初めて行われる



ハンズオン手順

ノートブック実行の流れ

- Git フォルダで pyspark_handson を最新へ Pull
- notebooks/01_pyspark_handson を開く
- Serverless または DBR 15.4 LTS 以降にアタッチ
- ウィジェットでカタログ・スキーマ・件数を設定
- Part 1(初級) → Part 2(中級・Delta Lake)の順に実行

Part 1: 初級 — PySpark の基礎

PySpark / Apache Spark とは

- OSS の並列分散処理フレームワーク。スケールアウトで線形に高速化
- 高速の理由: オンメモリ / Tungsten / キャッシュ / 遅延評価
- Python から Spark を扱う API が PySpark
- Databricks は Spark 上に構築、spark と dbutils が初期化済み

2つのモデル: RDD と DataFrame

RDD (Spark Core)

コレクション操作で記述

非構造データに柔軟・行指向

Python では遅い

DataFrame (Spark SQL)

SQL ライク・列指向

Catalyst が自動最適化

JVM 実行で高速 → 基本はこちら

アーキテクチャと遅延評価

- Python と JVM を Py4J で橋渡し。DataFrame は JVM 実行で高速
- RDD / Python UDF は Python プロセスで実行 → pickling コスト大
- 変換 (select/filter/join/groupBy): 定義のみで計算しない
- アクション (count/collect/display/write) で初めて計算。explain で計画確認

SparkSession と DataFrame 作成

- 入口は `SparkSession: getOrCreate / getActiveSession`、`spark.conf` で設定
- 作成方法: 辞書 / リスト / Row / 値指定、`spark.table`・`spark.read`
- schema は DDL 文字列で簡潔に指定
- メタデータ: `printSchema`・`columns`・`dtypes`・`schema`

基本操作とデータ型イディオム

- 列: select・withColumn・cast・withColumnRenamed / 行
: filter・distinct・sort・limit
- 条件分岐: when / otherwise (CASE 相当)
- NULL: coalesce で代替値、na.drop / na.fill
- 日付: datediff・current_date、複雑な型: struct・collect_list・explode

データソースの読み書き

- 書き込み: `write.format(...).mode(overwrite/append).save / saveAsTable`
- 読み込み: `read.format(csv/json/parquet).load ▪ spark.table`
- 分析は Parquet / Delta (列指向・高速) が定番
- テーブル=定義、データ=ファイル。入出力は `/Volumes/` を使う

Part 2: 中級 – 実践とDelta Lake

結合・集計 と SQL 連携

結合・集計

join: inner/left/outer

groupBy + agg で集計

sum/max は F. か別名 import

SQL の4スタイル + collect

spark.sql / selectExpr / expr

filter文字列 / alias

collect は絞ってから

設計・テスト・動的処理

- DataFrame in / DataFrame out で関数化 → 再利用とテストが容易
- unittest で createDataFrame の期待値と collect を比較
- スキーマ(メタデータ)に基づき cast/to_date を動的適用
- df.columns から SELECT 文を動的生成

Delta Lake: 信頼性の高いテーブル運用

- Databricks のテーブルは既定で Delta Lake 形式 (ACID・タイムトラベル)
- DELETE / UPDATE / MERGE(upsert) を 1 文で実行 (全書き換え不要)
- DESCRIBE HISTORY で履歴、VERSION AS OF で過去の状態を参照
- OPTIMIZE / Z-ORDER / VACUUM、スキーマ強制・進化

パフォーマンスと pandas 連携

- DataFrame・組み込み関数を優先(UDF は最後の手段)
- 早めに filter/select、小さい表は broadcast join
- Parquet/Delta(列指向)、データ偏り(Skew)に注意、explain で計画確認
- 小さく集計→toPandas(Arrow で高速化)、createDataFrame(pdf)で戻す



databricks